Discover : ce que je dis à la présentation

Pour chaque partie : la question que Mme TIKPON peut poser, la réponse à dire avec mes mots, puis le fichier et la ligne exacte à montrer. Pas besoin de connaître la syntaxe : il suffit de savoir **qui appelle qui**.

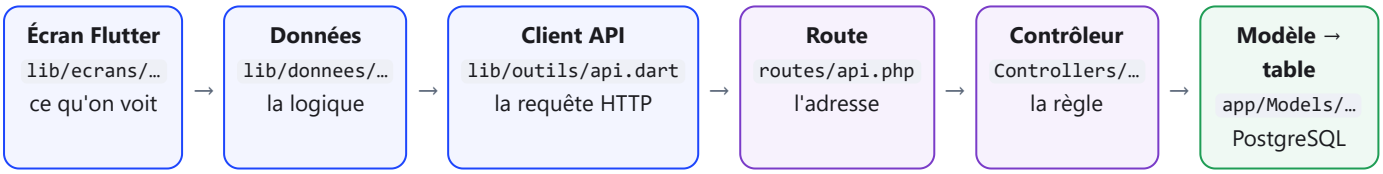
Dépôt cjdotdev/Discover, branche `simeon` · Application `usw_mobile` (Flutter) · Serveur `usw_api` (Laravel) · Version 2026

0. Comment utiliser ce guide

- **Les cadres bleus** sont des questions possibles. **Les cadres verts** sont les réponses à dire, presque mot pour mot.
- **Les tableaux « trajet »** suivent une action du doigt sur l'écran jusqu'à la base de données, dans l'ordre. Chaque ligne donne le fichier et le numéro de ligne : on ouvre le fichier dans VS Code, `Ctrl+G`, on tape le numéro, et on montre.
- Les étiquettes App Serveur Base disent de quel côté on est.
- Dans VS Code, `Ctrl+P` puis le nom du fichier (par exemple `session.dart`) l'ouvre tout de suite.

LA PHRASE D'OUVERTURE (30 SECONDES)

« Discover est une application Flutter de photos et de reels touristiques réels : chaque publication dit où et quand elle a été prise, et le bouton "Y aller" ouvre l'itinéraire. L'application ne touche jamais la base de données directement : elle envoie des requêtes HTTPS à notre API Laravel, hébergée sur Railway. C'est l'API qui vérifie qui parle, contrôle les données et lit ou écrit dans la base PostgreSQL, hébergée sur Neon. Les photos sont rangées sur Cloudflare R2, les e-mails partent par Resend et les notifications par Firebase. »



Toutes les fonctions de l'app suivent **ce même chemin**. Si je sais l'expliquer une fois, je sais l'expliquer pour tout.

1. Comment Flutter parle au serveur Laravel (la base de tout)

QUESTION POSSIBLE

« Comment votre application Flutter consomme-t-elle l'API ? »

CE QUE JE RÉPONDS

« Toutes les requêtes passent par **une seule classe**, `Api`, dans `lib/outils/api.dart`. Elle connaît l'adresse du serveur, ajoute à chaque requête le **jeton** du membre connecté, envoie et reçoit du **JSON**, et transforme toute erreur du serveur en une erreur Dart unique, `ErreurApi`, que les écrans affichent.

Les écrans n'appellent jamais le réseau eux-mêmes : ils appellent une classe de `lib/donnees` (par exemple `Session` pour le compte), qui appelle `Api`. Comme ça, si l'adresse du serveur change, on ne change qu'une ligne. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait

1	App **lib/outils/api.dart** ligne **54**	`static const base = String.fromEnvironment('API', defaultValue: 'https://discover-production-6cca.up.railway.app/api/v1');` L'adresse de l'API : le serveur Railway par défaut. On peut la changer au lancement avec `--dart-define=API=…` (par exemple pour un serveur local).
2	App **lib/outils/api.dart** ligne **141**	`Future<Reponse> get(String chemin) => _envoyer('GET', chemin);` Les raccourcis `get`, `post`, `put`, `patch`, `delete` : un par verbe HTTP. Les autres fichiers écrivent juste `Api.instance.post('auth/code', {…})`.
3	App **lib/outils/api.dart** ligne **212**	`final requete = await _client.openUrl(methode, Uri.parse('$base/$chemin')).timeout(delai);` La requête part vers `base/chemin`. On utilise `HttpClient`, livré avec Dart (`dart:io`).
4	App **lib/outils/api.dart** ligne **217**	`if (jeton != null) requete.headers.set(HttpHeaders.authorizationHeader, 'Bearer $jeton');` Si le membre est connecté, son jeton part dans l'en-tête `Authorization: Bearer …`. C'est ce que Laravel Sanctum lit pour savoir qui parle.
5	App **lib/outils/api.dart** ligne **220**	`final octets = brut ?? (corps == null ? null : utf8.encode(jsonEncode(corps)));` Le corps de la requête : un dictionnaire Dart transformé en JSON (`jsonEncode`).
6	App **lib/outils/api.dart** ligne **228**	`final json = texte.isEmpty ? <String, dynamic>{} : (jsonDecode(texte) as Map).cast<String, dynamic>();` La réponse JSON redevient un dictionnaire Dart (`jsonDecode`).
7	App **lib/outils/api.dart** ligne **231**	`if (reponse.statusCode == 401 && jeton != null) await garderJeton(null);` Code 401 : le jeton n'est plus valable, on l'oublie et le membre se reconnecte.
8	App **lib/outils/api.dart** ligne **236**	`throw ErreurApi(reponse.statusCode, json['message'] as String? ?? t('Une erreur s'est produite. Réessayez.'), erreurs);` Tout code 400 ou plus devient une `ErreurApi` avec le message du serveur, que l'écran affiche tel quel.
9	Serveur **bootstrap/app.php** ligne **17**	`apiPrefix: 'api/v1',` Côté Laravel : toutes les routes de `routes/api.php` commencent par `/api/v1`.

Pourquoi pas le paquet `http` ou `dio` ?

`HttpClient` est déjà dans Dart : pas de dépendance en plus, et il nous laisse régler le délai d'attente et les en-têtes nous-mêmes.

Que se passe-t-il sans réseau ?

Les erreurs réseau deviennent `ApiInjoignable` : l'écran affiche « Aucune connexion » au lieu de planter. Une lecture (GET) ratée est refaite une fois toute seule.

> **À retenir :** écran → classe de données → `Api` → route Laravel → contrôleur → modèle → table. Une seule porte de sortie vers le réseau : `api.dart`.

2. Se connecter avec un code reçu par e-mail

QUESTION POSSIBLE

« Comment avez-vous consommé l'API d'authentification faite avec Laravel dans Flutter ? Quel fichier intervient ? »

CE QUE JE RÉPONDS

« Il n'y a pas de mot de passe. Le membre tape son e-mail sur l'écran d'entrée (`entree.dart`). L'app appelle `Session.demanderCode` (`session.dart`), qui envoie un **POST** vers `auth/code/envoyer` . Côté Laravel, la route mène à `AuthController::envoyer` , qui tire un code à 6 chiffres, le range **haché** dans la table `codes_email` et l'envoie par e-mail.

Le membre tape le code (`code.dart`). L'app envoie un **POST** vers `auth/code` . Le contrôleur vérifie le code, retrouve le membre dans la table `utilisateurs` ou le crée, puis **Laravel Sanctum** crée un jeton. Le serveur renvoie le jeton et le profil en JSON. L'app garde le jeton sur le téléphone avec `SharedPreferences` , et l'envoie ensuite dans chaque requête. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/compte/entree.dart** ligne **54**	`await session.demanderCode(adresse);` L'écran d'entrée : bouton « Continuer », il demande un code pour l'adresse tapée.
2	App **lib/donnees/session.dart** ligne **131**	`final r = await Api.instance.post('auth/code/envoyer', {'email': email, 'defi': ?_defi});` La requête **POST auth/code/envoyer** avec l'e-mail.
3	Serveur **routes/api.php** ligne **33**	`Route::post('code/envoyer', [AuthController::class, 'envoyer'])->middleware('throttle:codes')->name('code.…` La route Laravel : cette adresse appelle la méthode `envoyer` du contrôleur. `throttle:codes` limite le nombre de codes par minute.
4	Serveur **Controllers/AuthController.php** ligne **31**	`public function envoyer(DemanderCode $requete): JsonResponse` Le contrôleur. `DemanderCode` est une Form Request : elle valide l'e-mail avant d'entrer ici.
5	Serveur **app/Services/Compte/GestionnaireCodes.php** ligne **52**	`$code = (string) random_int(100000, 999999);` Le code à six chiffres, tiré au hasard.
6	Base **app/Services/Compte/GestionnaireCodes.php** ligne **56**	`'code_hache' => Hash::make($code),` Il est rangé **haché** dans la table `codes_email` : même avec la base, on ne peut pas le lire.
7	Serveur **app/Services/Compte/GestionnaireCodes.php** ligne **62**	`Mail::to($email)->send(new CodeParEmail($code, $but));` L'e-mail part (service Resend).
8	App **lib/donnees/session.dart** ligne **132**	`_defi = r.json['defi'] as String? ?? _defi;` L'app garde le « défi » renvoyé : seul ce téléphone pourra essayer ce code.

9	App **lib/ecrans/compte/code.dart** ligne **85**	`final r = await s.verifierCode(_code, appareil: await modeleDuTelephone());` L'écran du code : les 6 chiffres tapés partent au serveur.
10	App **lib/donnees/session.dart** ligne **163**	`Future<Reponse> verifierCode(String code, {String? appareil}) => Api.instance.post('auth/code', {'email': email, '…` La requête **POST auth/code** avec l'e-mail, le code et le défi.
11	Serveur **routes/api.php** ligne **34**	`Route::post('code', [AuthController::class, 'code'])->middleware('throttle:connexion')->name('code');` La route vers la méthode `code`.
12	Serveur **app/Http/Requests/Compte/VerifierCode.php** ligne **12**	`'code' => ['required', 'string', 'digits:6'],` La validation : exactement 6 chiffres, sinon erreur 422 avant même le contrôleur.
13	Serveur **Controllers/AuthController.php** ligne **44**	`[$code, $echec, $reste] = $this->codes->verifier($email, $requete->validated('code'), GestionnaireCodes::BUT, $requete->valid…` Le code est comparé à l'empreinte (5 essais au plus).
14	Base **Controllers/AuthController.php** ligne **54**	`$u = Utilisateur::where('email', $email)->first();` Eloquent cherche le membre : `SELECT * FROM utilisateurs WHERE email = ?`. Absent : il est créé juste après (`$u->save()`).
15	Base **app/Services/Compte/Comptes.php** ligne **18**	`return $u->createToken(Str::limit($appareil ?: 'application', 60, ''))->plainTextToken;` Sanctum crée le jeton : une ligne dans `personal_access_tokens`. Le serveur ne garde qu'une empreinte du jeton.
16	Serveur **Controllers/AuthController.php** ligne **107**	`'jeton' => $this->comptes->jeton($u, $appareil),` La réponse JSON : `{ jeton, nouveau, utilisateur }`.
17	App **lib/donnees/session.dart** ligne **176**	`await Api.instance.garderJeton(r.json['jeton'] as String?);` L'app garde le jeton (`SharedPreferences`) : le membre est connecté.
18	App **lib/ecrans/compte/code.dart** ligne **28**	`if (context.mounted) context.go(s.profilComplete ? '/accueil' : '/completer/1');` Puis l'écran suivant : l'accueil, ou la fin du profil pour un nouveau compte.

Pourquoi un code plutôt qu'un mot de passe ?

Pas de mot de passe à retenir ni à protéger, et l'adresse est vérifiée du même coup. C'est le même parcours pour s'inscrire et pour se connecter.

Pourquoi hacher le code ?

Si quelqu'un lisait la base, il ne verrait que des empreintes. C'est le même principe que pour un mot de passe.

Qu'est-ce qu'une Form Request ?

Une classe Laravel qui contient les règles de validation. Si les données sont mauvaises, Laravel répond 422 tout seul, et le contrôleur reste court.

À retenir : deux requêtes : `POST auth/code/envoyer` (recevoir le code), puis `POST auth/code` (l'échanger contre un jeton Sanctum).

3. Se connecter avec Google

QUESTION POSSIBLE

« Et la connexion Google, comment marche-t-elle ? »

CE QUE JE RÉPONDS

« Dans Flutter, on utilise le paquet `google_sign_in`. Google affiche son choix de compte et nous donne un **jeton d'identité** (idToken). L'app ne le croit pas sur parole : elle l'envoie au serveur avec **POST auth/google**. Le serveur demande à Google si ce jeton est vrai, s'il est fait pour notre application et s'il n'a pas expiré. Si oui, il retrouve ou crée le membre et renvoie **notre propre** jeton Sanctum, exactement comme pour le code par e-mail. La suite est donc la même. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/compte/entree.dart** ligne **72**	`_googlePret ??= g.initialize(serverClientId: clientGoogle);` On prépare Google avec l'identifiant client de Discover.
2	App **lib/ecrans/compte/entree.dart** ligne **74**	`final jeton = (await g.authenticate()).authentication.idToken;` Google affiche le choix du compte et renvoie le jeton d'identité.
3	App **lib/ecrans/compte/entree.dart** ligne **76**	`final r = await Api.instance.post('auth/google', {'jeton': jeton, 'appareil': await modeleDuTelephone()});` Le jeton Google part au serveur, avec le modèle du téléphone.
4	Serveur **routes/api.php** ligne **35**	`Route::post('google', [AuthController::class, 'google'])->middleware('throttle:connexion')->name('google&#…` La route vers `AuthController::google`.
5	Serveur **Controllers/AuthController.php** ligne **67**	`$g = $verificateur->verifier($requete->validated('jeton'));` Le serveur fait vérifier le jeton.
6	Serveur **app/Services/Compte/VerificateurGoogle.php** ligne **21**	`$reponse = Http::timeout(8)->get('https://oauth2.googleapis.com/tokeninfo', ['id_token' => $jeton]);` Laravel demande à Google (`Http::get`) ce que contient le jeton.
7	Serveur **app/Services/Compte/VerificateurGoogle.php** ligne **27**	`$audienceOk = in_array($c['aud'] ?? '', $this->audiences, true);` Le jeton doit être fait pour **notre** application, sinon il est refusé.

8	Base **Controllers/AuthController.php** ligne **71**	`$u = Utilisateur::where('google_sub', $g['sub'])->first() ?? Utilisateur::where('email', $g['email'])-&g…` On retrouve le membre par son identifiant Google, sinon par son e-mail, sinon on le crée.
9	App **lib/ecrans/compte/entree.dart** ligne **78**	`await s.ouvrir(r);` Même réponse que pour le code : `Session.ouvrir` garde le jeton.

Pourquoi Google ne marche-t-il pas sur une app compilée chez un camarade ?
Google n'accepte que l'app signée avec notre clé de signature (son empreinte est déclarée chez Google). L'APK de l'équipe est signé ; une compilation faite ailleurs ne l'est pas. La connexion par e-mail marche partout.

> **À retenir :** Google prouve qui est la personne ; notre serveur vérifie cette preuve, puis donne **son** jeton Sanctum.

4. Rester connecté, être reconnu, se déconnecter

> **QUESTION POSSIBLE**
> *« Comment le serveur sait-il que c'est moi à chaque requête ? Que se passe-t-il quand je rouvre l'app ? »*

> **CE QUE JE RÉPONDS**
> « Au démarrage, `main.dart` appelle `Session.charger`, qui relit le jeton gardé sur le téléphone : l'app s'ouvre déjà connectée. Ensuite, chaque requête porte ce jeton. Côté Laravel, les routes protégées ont le middleware `auth:sanctum` : il cherche le jeton dans la table `personal_access_tokens` et retrouve le membre. On le récupère dans le contrôleur avec `$requete->user()`. Pour se déconnecter, l'app appelle **POST auth/deconnexion** et le serveur supprime le jeton de ce téléphone seulement. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/main.dart** ligne **46**	`await Session.instance.charger();` Au lancement de l'app : la session est rechargée.
2	App **lib/outils/api.dart** ligne **123**	`jeton = p.getString(_cleJeton);` Le jeton est relu depuis `SharedPreferences`.
3	App **lib/donnees/session.dart** ligne **104**	`final r = await Api.instance.get('moi');` **GET moi** : le profil à jour, d'après le serveur.
4	App **lib/donnees/session.dart** ligne **113**	`if (e.code == 401) _vider();` Si le serveur répond 401 (jeton refusé), tout est oublié : retour à l'écran d'entrée.
5	Serveur **routes/api.php** ligne **40**	`Route::middleware('auth:sanctum')->prefix('moi')->name('moi.')->group(function () {` Le groupe de routes `moi/…` : toutes exigent un jeton valable (middleware `auth:sanctum`).

6	Serveur **app/Models/Utilisateur.php** ligne **30**	`use HasApiTokens, HasFactory, HasUlids, Notifiable;` Le trait Sanctum `HasApiTokens` sur le modèle : il donne `createToken()` au membre.
7	Base **database/migrations/2026_10_23_095619_create_personal_access_tokens_table.php** ligne **14**	`Schema::create('personal_access_tokens', function (Blueprint $table) {` La table des jetons, créée par une migration.
8	App **lib/donnees/session.dart** ligne **265**	`await Api.instance.post('auth/deconnexion');` Déconnexion : **POST auth/deconnexion**.
9	Serveur **Controllers/AuthController.php** ligne **89**	`$jeton->delete();` Le serveur supprime le jeton de ce téléphone (les autres téléphones restent connectés).

À retenir : le jeton est gardé sur le téléphone et envoyé à chaque requête ; `auth:sanctum` le vérifie côté Laravel.

5. L'accueil : les lieux et leurs photos

QUESTION POSSIBLE

« D'où viennent les photos de l'accueil ? »

CE QUE JE RÉPONDS

« L'écran d'accueil (`accueil.dart`) demande les lieux à `Catalogue` (`catalogue.dart`), qui envoie **GET flux/decouverte**. Le contrôleur `ExplorationController::decouverte` lit les lieux et leurs meilleures photos dans PostgreSQL, 20 par page, et garde la page 60 secondes en cache. Il renvoie du JSON avec un **curseur** : quand le membre arrive en bas, l'app demande la page suivante avec ce curseur. Les images elles-mêmes sont téléchargées depuis Cloudflare R2, à l'adresse donnée dans le JSON. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/accueil.dart** ligne **61**	`onRefresh: Catalogue.instance.rafraichir,` Tirer l'écran vers le bas recharge l'accueil.
2	App **lib/ecrans/accueil.dart** ligne **53**	`if (n.metrics.axis == Axis.vertical && n.metrics.extentAfter < 900) Catalogue.instance.suite();` Arrivé presque en bas : la page suivante.
3	App **lib/donnees/catalogue.dart** ligne **275**	`final r = await Api.instance.get('flux/decouverte');` **GET flux/decouverte** : la première page.
4	App **lib/donnees/catalogue.dart** ligne **328**	`final r = await Api.instance.get('flux/decouverte?curseur=${Uri.encodeQueryComponent(_curseur!)}');` Les pages suivantes, avec le curseur.

5	Serveur **routes/api.php** ligne **98**	`Route::get('flux/decouverte', [ExplorationController::class, 'decouverte'])->name('flux.decouverte');` Route ouverte aux visiteurs : on peut regarder sans compte.
6	Serveur **Controllers/ExplorationController.php** ligne **63**	`public function decouverte(FluxDecouverte $requete): JsonResponse` Le contrôleur.
7	Serveur **Controllers/ExplorationController.php** ligne **67**	`$donnees = Cache::remember($cle, 60, function () use ($types, $requete) {` 60 secondes de cache : 100 visiteurs en une minute = une seule lecture de la base.
8	Base **Controllers/ExplorationController.php** ligne **70**	`->orderByDesc('score')->orderByDesc('nombre_contenus')->orderByDesc('id')->cursorPaginate(20, cursorName:…` Eloquent lit 20 lieux, triés par score, et calcule le curseur.
9	App **lib/ecrans/accueil.dart** ligne **127**	`listenable: Listenable.merge([MesPublications.instance, Catalogue.instance]),` Quand les données arrivent, `Catalogue` appelle `notifyListeners()` et ce `ListenableBuilder` redessine l'écran.

Comment l'écran sait-il que les données sont arrivées ?

Les classes de données (`Session`, `Catalogue`…) héritent de `ChangeNotifier`. Elles appellent `notifyListeners()`, et les `ListenableBuilder` qui les écoutent se redessinent.

Pourquoi un curseur plutôt que des numéros de page ?

Si une photo est publiée pendant qu'on fait défiler, les numéros de page décalent tout ; le curseur dit « après cet élément-là », donc rien n'est sauté ni répété.

> **À retenir :** `GET flux/decouverte` → `ExplorationController::decouverte` → PostgreSQL ; l'écran se redessine grâce à `ChangeNotifier`.

6. La carte et « Y aller »

> **QUESTION POSSIBLE**
>
> *« Comment fonctionne la carte ? »*

> **CE QUE JE RÉPONDS**
>
> « La carte est celle de Google (`google_maps_flutter`), dans `carte_google.dart`. Quand le membre arrête de bouger la carte, l'app lit les quatre bords de la zone visible (nord, sud, est, ouest) et envoie **GET carte?nord=…&sud=…&est=…&ouest=…**. Le serveur renvoie les lieux qui se trouvent dans ce rectangle, 300 au plus, et l'app place un marqueur pour chacun. La clé Google Maps n'est pas dans le code : elle est lue dans `cles.json` au moment de la compilation. « Y aller » n'appelle pas notre serveur : il ouvre Google Maps avec les coordonnées du lieu. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait

1	App **lib/widgets/carte_google.dart** ligne **93**	`GoogleMap(` Le widget de la carte Google.
2	App **lib/widgets/carte_google.dart** ligne **110**	`onCameraIdle: widget.onArret,` La carte s'arrête de bouger : on prévient l'écran.
3	App **lib/ecrans/carte.dart** ligne **73**	`final b = await c.getVisibleRegion();` Les bords de la zone visible.
4	App **lib/ecrans/carte.dart** ligne **85**	`final lieux = await Catalogue.instance.lieuxDeLaZone(nord: z.nord, sud: z.sud, est: z.est, ouest: z.ouest, types: types);` L'écran demande les lieux de cette zone.
5	App **lib/donnees/catalogue.dart** ligne **543**	`final r = await Api.instance.get('carte?nord=${n(nord)}&sud=${n(sud)}&est=${n(est)}&ouest=${n(ouest)}$filtre');` **GET carte** avec les quatre bords.
6	Serveur **Controllers/ExplorationController.php** ligne **163**	`public function carte(ZoneDeCarte $requete): AnonymousResourceCollection` Le contrôleur ; `ZoneDeCarte` valide les coordonnées.
7	Base **Controllers/ExplorationController.php** ligne **97**	`->whereBetween('latitude', [$latMin, $latMax])` La requête SQL : les lieux dont la latitude et la longitude sont dans la zone.
8	Base **Controllers/ExplorationController.php** ligne **177**	`->orderByDesc('nombre_contenus')->limit(300)->get();` Les 300 lieux les plus fournis d'abord.
9	App **lib/ecrans/carte.dart** ligne **292**	`Marker(` Un marqueur par lieu reçu.
10	App **usw_mobile/android/app/build.gradle.kts** ligne **5**	`val cleGoogleMaps: String = rootProject.file("../cles.json").takeIf { it.exists() }?.let {` La clé Google Maps est lue dans `cles.json` (hors du dépôt public)…
11	App **usw_mobile/android/app/src/main/AndroidManifest.xml** ligne **66**	`android:name="com.google.android.geo.API_KEY"` …puis placée dans le manifeste Android, où le SDK Maps la cherche.
12	App **lib/widgets/y_aller.dart** ligne **64**	`final itineraire = Uri.parse('https://www.google.com/maps/dir/?api=1&destination=${lieu.latitude},${lieu.longitude}');` « Y aller » : un lien d'itinéraire vers les coordonnées du lieu, ouvert dans Google Maps.

À retenir : la carte envoie sa zone visible, le serveur renvoie les lieux de cette zone ; « Y aller » ouvre Google Maps sans passer par nous.

7. Aimer une publication (et prévenir l'auteur)

QUESTION POSSIBLE

« Quand je mets un j'aime, que se passe-t-il ? »

CE QUE JE RÉPONDS

« Le cœur change **tout de suite** à l'écran, sans attendre le serveur. En même temps, `Communaute.reagir` envoie **PUT contenus/{id}/reaction** (et **DELETE** pour retirer). Côté Laravel, `CommunauteController::reagir` enregistre la réaction dans la table `reactions` avec `updateOrCreate` : une seule réaction par membre et par publication. Puis il crée une notification pour l'auteur. Le serveur renvoie le nouveau total, que l'app affiche. Si le réseau est coupé, le geste est gardé et renvoyé plus tard. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/photo.dart** ligne **206**	`if (!await Communaute.instance.reagir(_publication.id, oui ? 'coeur' : null) && mounted) {` L'écran de la photo : le cœur touché.
2	App **lib/donnees/communaute.dart** ligne **246**	`_reactions[contenu] = type;` L'affichage change tout de suite (on n'attend pas le serveur).
3	App **lib/donnees/communaute.dart** ligne **249**	`final r = type == null ? await Api.instance.delete('contenus/$contenu/reaction') : await Api.instance.put('contenus/$contenu/reactio…` **PUT** pour mettre la réaction, **DELETE** pour la retirer.
4	Serveur **routes/api.php** ligne **129**	`Route::put('contenus/{contenu}/reaction', [CommunauteController::class, 'reagir'])->name('reactions.mettre');` La route : elle exige un compte (groupe `auth:sanctum`).
5	Serveur **Controllers/CommunauteController.php** ligne **43**	`public function reagir(Request $requete, Contenu $contenu): JsonResponse` Laravel trouve tout seul la publication grâce à `{contenu}` dans l'adresse (liaison de modèle).
6	Base **Controllers/CommunauteController.php** ligne **47**	`$r = Reaction::updateOrCreate(['utilisateur_id' => $requete->user()->id, 'contenu_id' => $contenu->id], ['t…` Une ligne dans `reactions` : créée, ou mise à jour si elle existait.
7	Serveur **Controllers/CommunauteController.php** ligne **50**	`Activite::a($contenu->auteur, new Activite('reaction', 'a réagi :emoji à votre publication.', $requete->user(), $contenu,…` Une notification « a réagi à votre publication » pour l'auteur.
8	App **lib/donnees/communaute.dart** ligne **251**	`final n = (r.json['reactions'] as num?)?.toInt();` Le nombre exact renvoyé par le serveur remplace celui de l'écran.

À retenir : l'écran réagit tout de suite, la requête `PUT` suit ; le serveur enregistre et prévient l'auteur.

8. Commenter

QUESTION POSSIBLE

« Et les commentaires ? »

CE QUE JE RÉPONDS

« La feuille des commentaires (`feuilles.dart`) appelle `Communaute.commenter`, qui envoie **POST contenus/{id}/commentaires** avec le texte. `CommentaireController::creer` valide le texte, crée la ligne dans la table `commentaires`, puis prévient l'auteur de la publication (et la personne à qui on répond, ou qu'on mentionne avec @). La liste se lit avec **GET contenus/{id}/commentaires**. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/widgets/feuilles.dart** ligne **371**	`.commenter(widget.contenu!, texte, parent: aRepondre?.id)` Le bouton « Envoyer » de la feuille des commentaires.
2	App **lib/donnees/communaute.dart** ligne **403**	`final r = await Api.instance.post('contenus/$contenu/commentaires', {'texte': texte, 'parent': ?parent});` **POST** avec le texte (et le commentaire parent pour une réponse).
3	Serveur **routes/api.php** ligne **132**	`Route::post('contenus/{contenu}/commentaires', [CommentaireController::class, 'creer'])->middleware('throttle:commenter…` La route, avec une limite de débit (`throttle:commenter`).
4	Serveur **Controllers/CommentaireController.php** ligne **61**	`public function creer(Request $requete, Contenu $contenu): JsonResponse` Le contrôleur.
5	Base **Controllers/CommentaireController.php** ligne **87**	`$c = Commentaire::create([` La ligne dans `commentaires` .
6	Serveur **Controllers/CommentaireController.php** ligne **109**	`Activite::a($auteur, new Activite('commentaire', 'a commenté votre publication.', $moi, $contenu, $c));` La notification pour l'auteur de la publication.
7	App **lib/donnees/communaute.dart** ligne **381**	`final r = await Api.instance.get('contenus/$contenu/commentaires');` La lecture de la liste : **GET**.

À retenir : `POST` pour écrire, `GET` pour lire, sur la même adresse `contenus/{id}/commentaires` .

9. Le profil d'un créateur et « Suivre »

QUESTION POSSIBLE

« Comment s'affiche un profil, et comment marche l'abonnement ? »

CE QUE JE RÉPONDS

« L'écran du profil (`profil.dart`) appelle `Catalogue.chargerCreateur` , qui envoie **GET utilisateurs/{nom}** puis **GET utilisateurs/{nom}/contenus** pour ses publications. Le bouton « Suivre » appelle `Communaute.suivre` , qui envoie **PUT utilisateurs/{nom}/abonnement** (DELETE pour ne plus suivre). Le serveur refuse qu'on se suive soi-même, crée la ligne dans la table `abonnements` et prévient le créateur. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/profil.dart** ligne **710**	`Catalogue.instance.chargerCreateur(widget.pseudo).then((c) {` L'écran du profil demande le créateur.
2	App **lib/donnees/catalogue.dart** ligne **586**	`final r = await Api.instance.get('utilisateurs/${Uri.encodeComponent(p)}');` **GET utilisateurs/{nom}** (et ses contenus juste après).
3	Serveur **Controllers/UtilisateurController.php** ligne **25**	`public function afficher(string $nom): CreateurResource` Le contrôleur renvoie le profil public (`CreateurResource`).
4	App **lib/ecrans/profil.dart** ligne **693**	`final n = await Communaute.instance.suivre(widget.pseudo, oui);` Le bouton « Suivre ».
5	App **lib/donnees/communaute.dart** ligne **518**	`final r = oui ? await Api.instance.put('utilisateurs/$p/abonnement', {}) : await Api.instance.delete('utilisateurs/$p/abonnement…` **PUT** pour suivre, **DELETE** pour arrêter.
6	Serveur **Controllers/CommunauteController.php** ligne **106**	`public function sAbonner(Request $requete, string $nom): JsonResponse` Le contrôleur : pas soi-même, pas un compte bloqué.
7	Base **Controllers/CommunauteController.php** ligne **116**	`$a = Abonnement::firstOrCreate(['abonne_id' => $moi->id, 'createur_id' => $createur->id]);` La ligne dans `abonnements` (une seule par paire).
8	Serveur **Controllers/CommunauteController.php** ligne **118**	`Activite::a($createur, new Activite('abonne', 's'est abonné à vous.', $moi));` La notification « s'est abonné à vous ».

À retenir : une **Resource** Laravel décide quels champs du membre partent en JSON : jamais l'e-mail d'un autre, par exemple.

10. La recherche

QUESTION POSSIBLE

« Comment marche la recherche ? »

CE QUE JE RÉPONDS

« L'écran de recherche (`recherche.dart`) appelle `Catalogue.chercher`, qui envoie **GET recherche?q=…**. Le contrôleur cherche des lieux (par leur nom ou leur ville) et des créateurs, en ignorant les accents et les majuscules, 10 de chaque au plus, et renvoie le tout en JSON. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/recherche.dart** ligne **158**	`final r = await Catalogue.instance.chercher(q);` Le texte tapé.
2	App **lib/donnees/catalogue.dart** ligne **558**	`final r = await Api.instance.get('recherche?q=${Uri.encodeQueryComponent(q)}');` **GET recherche?q=…** ; `Uri.encodeQueryComponent` protège les espaces et les accents dans l'adresse.
3	Serveur **routes/api.php** ligne **105**	`Route::get('recherche', [ExplorationController::class, 'recherche'])->name('recherche');` Route ouverte aux visiteurs.
4	Serveur **Controllers/ExplorationController.php** ligne **192**	`public function recherche(Chercher $requete): JsonResponse` Le contrôleur ; `Chercher` valide le texte.

11. Publier une photo

QUESTION POSSIBLE

« Comment une photo va-t-elle du téléphone au serveur ? »

CE QUE JE RÉPONDS

« En trois temps. D'abord **POST contenus** : le serveur crée un brouillon et deux « envois » (l'aperçu et l'original). Ensuite le fichier part **par morceaux** avec **PATCH envois/{id}** ; avant chaque reprise, l'app demande **HEAD envois/{id}** « où en es-tu ? », donc une coupure de réseau ne fait pas tout recommencer. Enfin **POST envois/{id}/terminer** : le serveur met l'analyse dans la **file d'attente** (vérifier que c'est une vraie photo, créer les tailles WebP, les ranger sur R2), répond tout de suite, et prévient les abonnés une fois la photo publiée. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/ecrans/creer.dart** ligne **1483**	`Envois.instance.publier(titre: titre, photo: couverture, poidsMo: poids, source: source);` Le bouton « Publier » de l'écran de création.
2	App **lib/donnees/envois.dart** ligne **497**	`final r = await Api.instance.post('contenus', s.corps(fichiers));` **POST contenus** : le titre, le lieu, la taille des fichiers.

3	Serveur **Controllers/ContenuController.php** ligne **65**	`public function creer(CreerContenu $requete, Publication $publication): JsonResponse` Le contrôleur crée le brouillon et ses envois (service `Publication`).
4	App **lib/donnees/envois.dart** ligne **565**	`final etat = await Api.instance.head('envois/$id');` **HEAD** : combien d'octets le serveur a déjà reçus.
5	App **lib/donnees/envois.dart** ligne **575**	`final r = await Api.instance.morceau('envois/$id', decalage, octets);` **PATCH** : un morceau, avec son décalage (en-tête `Upload-Offset`).
6	Serveur **app/Services/Envois/Reception.php** ligne **41**	`if ($decalage !== $envoi->recu) {` Un morceau qui ne suit pas le précédent est refusé (409) : le fichier ne peut pas être mélangé.
7	App **lib/donnees/envois.dart** ligne **589**	`await Api.instance.post('envois/$id/terminer');` **POST terminer** : le fichier est complet.
8	Serveur **Controllers/EnvoiController.php** ligne **64**	`AnalyserCapture::dispatch($contenu);` `dispatch` met l'analyse dans la file d'attente : la réponse part tout de suite.
9	Serveur **app/Jobs/AnalyserCapture.php** ligne **117**	`PrevenirAbonnes::dispatch($c);` Publiée : une autre tâche prévient les abonnés.

À retenir : brouillon (`POST contenus`) → morceaux (`HEAD` + `PATCH`) → `terminer` → file d'attente.

12. Les notifications push

QUESTION POSSIBLE

« Comment recevez-vous les notifications sur le téléphone ? »

CE QUE JE RÉPONDS

« On utilise **Firebase Cloud Messaging**. Au démarrage, l'app initialise Firebase, demande la permission, puis récupère le **jeton Firebase** du téléphone. Elle l'envoie au serveur avec **PUT moi/push** ; Laravel le range dans la table `appareils_push`. Quand quelqu'un aime, commente ou s'abonne, le serveur crée une notification Laravel : elle est enregistrée dans la base (l'écran des notifications) **et** envoyée au téléphone par notre canal `CanalPush`, qui passe par la file d'attente, puis par l'API de Firebase. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	App **lib/main.dart** ligne **49**	`Push.instance.demarrer().ignore();` Au lancement.
2	App **lib/donnees/push.dart** ligne **65**	`await Firebase.initializeApp();` Firebase démarre (il lit `google-services.json`, hors du dépôt public).

3	App **lib/donnees/push.dart** ligne **118**	`final reglage = await m.requestPermission();` Android 13 et plus demandent la permission, une fois.
4	App **lib/donnees/push.dart** ligne **121**	`final jeton = _jeton ??= await m.getToken();` Le jeton Firebase de ce téléphone.
5	App **lib/donnees/push.dart** ligne **123**	`await Api.instance.put('moi/push', {'jeton': jeton, 'preferences': choix, 'pause_jusqu_a': enPause ? pauseJ…` **PUT moi/push** : le jeton et les réglages.
6	Base **Controllers/MoiController.php** ligne **283**	`AppareilPush::updateOrCreate(['jeton' => $v['jeton']], [` Rangé dans `appareils_push`, relié au membre.
7	Serveur **app/Notifications/Activite.php** ligne **42**	`return ['database', CanalPush::class];` Une notification part par deux canaux : la base (table `notifications`) et le push.
8	Serveur **app/Notifications/Canaux/CanalPush.php** ligne **30**	`EnvoyerPush::dispatch($appareil->id, $m['titre'], $m['corps'], $m['donnees'], $m['image'] ?? null);` Une tâche en file par téléphone du membre.
9	Serveur **app/Services/Push/Firebase.php** ligne **39**	`->post("https://fcm.googleapis.com/v1/projects/{$compte['project_id']}/messages:send", [` L'envoi à Firebase, qui le livre au téléphone.
10	App **lib/donnees/push.dart** ligne **77**	`FirebaseMessaging.onMessageOpenedApp.listen((_) => _touchee());` La notification touchée ouvre l'écran des notifications.
11	App **lib/donnees/communaute.dart** ligne **611**	`final r = await Api.instance.get('notifications${curseur == null ? '' : '?curseur=${Uri.encodeQueryComponent(curseur)}'}&#…` Cet écran lit la liste avec **GET notifications**.

À retenir : le téléphone donne son jeton Firebase au serveur ; le serveur s'en sert pour lui envoyer les notifications.

13. La base de données

QUESTION POSSIBLE

« Comment le serveur est-il relié à la base ? Comment avez-vous créé les tables ? »

CE QUE JE RÉPONDS

« La base est PostgreSQL, hébergée sur **Neon**. Laravel s'y connecte avec une adresse rangée dans la variable d'environnement `DB_URL` sur Railway : elle n'est ni dans le code, ni dans Git. La connexion est chiffrée (SSL). Les tables sont créées par des **migrations** (30 fichiers dans `database/migrations`), appliquées automatiquement à chaque déploiement. Dans le code, on n'écrit presque pas de SQL : on passe par les **modèles Eloquent**, une classe PHP par table, et Laravel traduit en SQL. »

Le trajet dans le code, à montrer si elle demande « où ? »

#	Où	La ligne et ce qu'elle fait
1	Serveur **config/database.php** ligne **38**	`'url' => env('DB_URL'),` L'adresse de la base vient de l'environnement de Railway.
2	Serveur **config/database.php** ligne **117**	`'sslmode' => env('DB_SSLMODE', 'require'),` Connexion chiffrée obligatoire.
3	Base **database/migrations/2026_10_24_170000_create_communaute_tables.php** ligne **17**	`Schema::create('reactions', function (Blueprint $table) {` Exemple de migration : la table `reactions` décrite en PHP.
4	Base **app/Models/Utilisateur.php** ligne **32**	`protected $table = 'utilisateurs';` Exemple de modèle : `Utilisateur` correspond à la table `utilisateurs`.
5	Serveur **docker/demarrer.sh** ligne **12**	`DB_URL="${DB_URL_DIRECTE:-$DB_URL}" php artisan migrate --force` Au démarrage du serveur sur Railway, les migrations sont appliquées.
6	Serveur **app/Http/Resources/ContenuResource.php** ligne **21**	`'id' => $this->ulid,` Une Resource transforme une ligne en JSON. L'identifiant public est un ULID, pas le numéro de ligne : on ne peut pas deviner les autres.

Les tables, par rôle

Rôle	Tables
Comptes et connexion	utilisateurs, personal_access_tokens (jetons Sanctum), codes_email, appareils_push
Lieux	pays, villes, localites, lieux, lieux_enregistres
Publications	contenus, captures, modeles_telephone, fichiers_originaux, variantes, envois, etiquettes
Communauté	reactions, commentaires, jaimes_commentaires, collections, enregistrements, abonnements, notifications, vues, desinterets
Modération	signalements, blocages, decisions_moderation
Fonctionnement de Laravel	cache, jobs, failed_jobs, sessions

> **À retenir :** migrations = la forme des tables ; modèles Eloquent = lire et écrire dedans ; Resources = ce qui part en JSON.

14. L'hébergement et la mise en ligne

> **QUESTION POSSIBLE**
>
> *« Où tourne votre serveur ? »*

CE QUE JE RÉPONDS

« Le serveur Laravel tourne sur **Railway**, dans un conteneur Docker (`usw_api/Dockerfile`). Au démarrage, le script `docker/demarrer.sh` applique les migrations, puis lance le serveur web et, dans le même conteneur, l'ouvrier de la file d'attente (`queue:work`) qui fait les tâches longues. Les secrets (base, R2, Resend, Firebase, clé Laravel) sont des variables d'environnement de Railway. L'app Flutter, elle, est un APK Android signé par l'équipe. »

À retenir : Railway = le serveur ; Neon = la base ; R2 = les fichiers ; Firebase = les notifications ; Resend = les e-mails.

15. Les questions pièges, réponses courtes

Pourquoi l'app ne parle-t-elle pas directement à la base ?

Il faudrait mettre le mot de passe de la base dans l'app, et n'importe qui pourrait le lire en décompilant l'APK. Le serveur garde les secrets, vérifie les droits et valide les données.

Qu'est-ce qu'une API REST ?

Des adresses (routes) et des verbes HTTP : GET pour lire, POST pour créer, PUT/PATCH pour modifier, DELETE pour supprimer. Les données circulent en JSON.

Qu'est-ce que Sanctum ?

Le système de Laravel qui crée des jetons pour les applications mobiles. Le jeton remplace la session d'un site web.

Qu'est-ce qu'un middleware ?

Un contrôle placé devant les routes : `auth:sanctum` vérifie le jeton, `throttle` limite le nombre de requêtes.

Que veulent dire 401, 403, 422 ?

401 : pas connecté (jeton absent ou refusé). 403 : connecté, mais pas le droit. 422 : données invalides ; l'app affiche le message sous le bon champ.

Comment empêchez-vous quelqu'un de modifier la publication d'un autre ?

Les Policies de Laravel (`ContenuPolicy`) vérifient que le membre est bien l'auteur avant de modifier ou supprimer.

Pourquoi une file d'attente ?

Analyser une photo prend plusieurs secondes. Le membre reçoit sa réponse tout de suite, et un ouvrier fait le travail en fond.

Où sont les mots de passe ?

Dans les variables d'environnement de Railway, jamais dans le code ni dans Git. Le fichier `.env.example` ne donne que leurs noms.

Que se passe-t-il si le réseau coupe ?

L'app affiche « Aucune connexion » au lieu de planter. Un j'aime fait hors ligne est gardé et renvoyé. Un envoi de photo reprend où il s'était arrêté.

Comment gérez-vous l'état dans Flutter ?

Avec `ChangeNotifier` : une instance unique par domaine (`Session`, `Catalogue`, `Communaute`) ; les écrans l'écoutent avec `ListenableBuilder`.

Comment passez-vous d'un écran à l'autre ?

Avec `go_router` (fichier `lib/app.dart`) : chaque écran a une adresse, par exemple `/accueil` ou `/entree/code`.

16. Toutes les liaisons sur une page

À gauche, la ligne de l'app qui envoie la requête ; à droite, la ligne de `routes/api.php` qui la reçoit et le contrôleur qui la traite.

Action	App : ligne	Requête	Serveur : ligne
Demander un code	`lib/donnees/session.dart` : 131	`POST auth/code/envoyer`	`routes/api.php` : 33 AuthController::envoyer
Vérifier le code	`lib/donnees/session.dart` : 163	`POST auth/code`	`routes/api.php` : 34 AuthController::code
Google	`lib/ecrans/compte/entree.dart` : 76	`POST auth/google`	`routes/api.php` : 35 AuthController::google
Se déconnecter	`lib/donnees/session.dart` : 265	`POST auth/deconnexion`	`routes/api.php` : 36 AuthController::deconnexion
Mon profil	`lib/donnees/session.dart` : 104	`GET moi`	`routes/api.php` : 41 MoiController::afficher
Modifier mon profil	`lib/donnees/session.dart` : 191	`PATCH moi`	`routes/api.php` : 42 MoiController::modifier
Accueil	`lib/donnees/catalogue.dart` : 275	`GET flux/decouverte`	`routes/api.php` : 98 ExplorationController::decouverte
Reels	`lib/donnees/catalogue.dart` : 272	`GET reels`	`routes/api.php` : 100 ExplorationController::reels
Un lieu	`lib/donnees/catalogue.dart` : 378	`GET lieux/{lieu}`	`routes/api.php` : 69 LieuController::afficher
Une publication	`lib/donnees/catalogue.dart` : 430	`GET contenus/{contenu}`	`routes/api.php` : 71 ContenuController::afficher
Carte	`lib/donnees/catalogue.dart` : 543	`GET carte`	`routes/api.php` : 104 ExplorationController::carte
Recherche	`lib/donnees/catalogue.dart` : 558	`GET recherche`	`routes/api.php` : 105 ExplorationController::recherche
Profil d'un créateur	`lib/donnees/catalogue.dart` : 586	`GET utilisateurs/{nom}`	`routes/api.php` : 110 UtilisateurController::afficher
Réagir	`lib/donnees/communaute.dart` : 249	`PUT contenus/{contenu}/reaction`	`routes/api.php` : 129 CommunauteController::reagir
Commenter	`lib/donnees/communaute.dart` : 403	`POST contenus/{contenu}/commentaires`	`routes/api.php` : 132 CommentaireController::creer
Suivre	`lib/donnees/communaute.dart` : 518	`PUT utilisateurs/{nom}/abonnement`	`routes/api.php` : 139 CommunauteController::sAbonner
Collections	`lib/donnees/communaute.dart` : 451	`GET collections`	`routes/api.php` : 146 CollectionController::index
Notifications	`lib/donnees/communaute.dart` : 611	`GET notifications`	`routes/api.php` : 156 NotificationController::index

Créer une publication	`lib/donnees/envois.dart` : 497	`POST contenus`	`routes/api.php` : 85 ContenuController::creer
Envoyer un morceau	`lib/donnees/envois.dart` : 575	`PATCH envois/{envoi}`	`routes/api.php` : 92 EnvoiController::morceau
Terminer l'envoi	`lib/donnees/envois.dart` : 589	`POST envois/{envoi}/terminer`	`routes/api.php` : 93 EnvoiController::terminer
Recevoir les push	`lib/donnees/push.dart` : 123	`PUT moi/push`	`routes/api.php` : 49 MoiController::push
Langues	`lib/outils/langue.dart` : 157	`GET langues`	`routes/api.php` : 108 LangueController::index

17. Les mots à connaître

Mot	Ce que ça veut dire, simplement
API	L'ensemble des adresses que le serveur met à disposition de l'app.
Route	Une adresse + un verbe (ex. `POST auth/code`), reliée à une méthode de contrôleur.
Contrôleur	La classe PHP qui reçoit la requête, applique la règle et renvoie la réponse.
Modèle (Eloquent)	Une classe PHP qui représente une table : `Utilisateur` ↔ `utilisateurs`.
Migration	Un fichier PHP qui crée ou modifie une table. On peut recréer la base n'importe où.
JSON	Le format texte des échanges : `{"jeton": "…", "utilisateur": {…}}`.
Jeton (Sanctum)	Une longue chaîne donnée à la connexion. L'app la renvoie à chaque requête pour dire « c'est moi ».
Middleware	Un contrôle avant la route : être connecté, ne pas faire trop de requêtes.
Future / async / await	En Dart, une requête prend du temps : `await` attend la réponse sans bloquer l'écran.
ChangeNotifier	Une classe Flutter qui prévient les écrans quand ses données changent (`notifyListeners`).
SharedPreferences	Un petit stockage sur le téléphone : on y garde le jeton.
File d'attente (queue)	Des tâches longues mises de côté et faites en fond par un « ouvrier ».